

# NASA cFS user-app and Custom Linux Driver

By: Md Ashfaaq Khan

## Summary & Implementation Architecture

This implementation establishes an end-to-end telemetry and command pipeline bridging an isolated, unprivileged user-space flight software environment (NASA cFS) with a privileged Linux Kernel space driver simulating hardware registers (Enable Register at offset 0x00, Status Register at offset 0x04).

The architecture is implemented across three subsystems:

The Kernel Driver (pixxel\_platform\_driver): A loadable kernel module that exposes a virtual communication interface at `/dev/pixxel`. It uses internal kernel structures to mimic real hardware delay constraints. (status\_reg and enable\_reg)

A few resources used to build it:

<https://docs.kernel.org/driver-api/basics.html>

<https://www.kernel.org/doc/html/latest/driver-api/driver-model/platform.html>

<https://stackoverflow.com/questions/34704739/i-need-help-in-understanding-the-following-kernel-module-written-in-c>

The Pixxel main (PIXXEL\_MAIN): A high-level flight application that drives the execution timeline: commanding the device on, enforcing stabilization delays, and verifying status confirmation via standard cFS software telemetry buffers.

The Pixxel controller (PIXXEL\_CTRL): Built within the cFS framework, this application acts as the mediator. It reads asynchronous commands from the flight system Software Bus and maps them to synchronous VFS operations (open, write, read) targeting the driver node.

The cFS readmes and sample app files and headers were very helpful to write this along with a few tutorials: <https://davefer.blogspot.com/2024/02/cfs-tutorial-part-3-putting-messages.html>

[https://github.com/nasa/sample\\_app/blob/dev/fsw/src/sample\\_app.c](https://github.com/nasa/sample_app/blob/dev/fsw/src/sample_app.c)

<https://davefer.blogspot.com/2023/05/nasa-cfs-tutorial-stage-ii-transform.html>

**Could not implement yocto due to hardware limitations (yocto required 50GB free space atleast but my partition was itself 60GB)**

**Though the code logic of the driver and the cFS user apps were correct, there was physically no way to emulate them on my machine**

➤ df -h ~

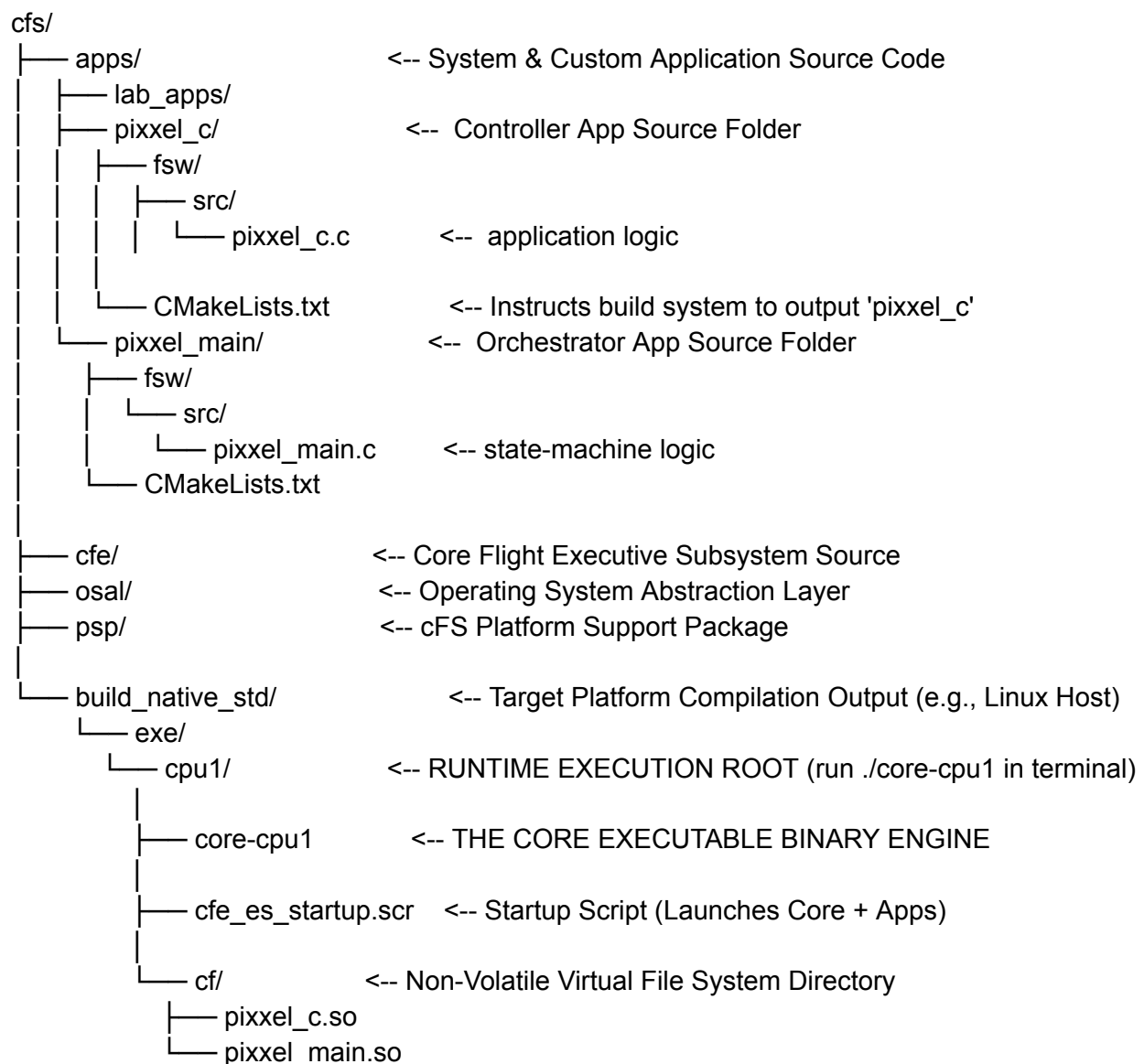
| Filesystem     | Size | Used | Avail | Use% | Mounted on |
|----------------|------|------|-------|------|------------|
| /dev/nvme0n1p4 | 62G  | 50G  | 8.5G  | 86%  | /          |

## Implementation Architecture:

After compiling the `pixxel_platform_driver.c` to get to `pixxel_platform_driver.ko` and using `insmod` to insert into the kernel on native (Linux g14 7.0.5-arch1-1-g14) machine and receiving no errors, I implemented a test app to interact with the platform\_driver to verify working. Since my native dtb does not the `pixxel_virt_dev`, the only output is `pixxel_device not found`.

The cFS sample app and the tutorial helped a lot with the compiling and running of the apps. Running the `./core-cpu1` with `grep` to only display the required outputs after editing the startup script with the apps we see that they exit cause nothing exists at `dev/pixxel`

The file system for the cFS.



# Challenges

## Hardware Architecture vs. Virtual Register Emulation

Interfacing with actual physical hardware requires utilizing memory-mapped physical registers via `ioread32()` and `iowrite32()` hooks. However, because those hardware API macros explicitly expect a memory-mapped base address (`void __iomem *`) generated by `ioremap()`, invoking them or internal software tracking variables (`static u32`) causes immediate kernel faults or undefined side effects due to mismatched bus architectures.

Explained a little better here

<https://stackoverflow.com/questions/26831583/ioread32-followed-by-iowrite32-not-giving-same-value>

For this virtual emulation layer, the implementation purposely utilizes direct software state tracking variables (`enable_reg`, `status_reg`). To align this architecture seamlessly with future hardware drop-in readiness, these local software assignments are fully decoupled inside the read/write syscall handlers.

## The cFS OSAL Boot Failure (EC = -104)

During startup, the engine failed with: `CFE_ES_LoadModule: Could not load file:/cf/pixxel_controller.so. EC = -104`, causing an immediate system shutdown.

```
#define OS_FS_ERR_NAME_TOO_LONG    (-104) /**< @brief FS name too long */
```

In the cFS OSAL header (`osapi-error.h`), error -104 is defined as `OS_FS_ERR_NAME_TOO_LONG`. The application name `pixxel_controller` exceeded the fixed character buffer sizes allocated inside the cFS startup script parser.

Solution: Refactored `CMakeLists.txt`, `cfe_es_startup.scr`, and source targets to use a compressed 8-character string: **pixxel\_c**. This satisfied the internal boundary constraints and allowed successful runtime injection.

## The working outputs

```
sudo insmod pixxel_platform_driver.ko
```

```
➤ lsmod
```

| Module                              | Size  | Used by |
|-------------------------------------|-------|---------|
| <code>pixxel_platform_driver</code> | 20480 | 0       |
| <code>uinput</code>                 | 28672 | 1       |
| <code>ccm</code>                    | 24576 | 6       |

For the pixxel\_main and pixxel\_c

```
./core-cpu1 2>&1 | grep -E "PIXXEL|pixxel|Pixxel"
```

1980-012-14:03:20.50262 CFE\_ES\_ParseFileEntry: Loading file: /cf/pixxel\_c.so, APP:

PIXXEL\_CTRL

1980-012-14:03:20.50282 CFE\_ES\_ParseFileEntry: Loading file: /cf/pixxel\_main.so, APP:

PIXXEL\_MAIN

EVS Port1 1980-012-14:03:20.55278 66/1/PIXXEL\_CTRL 1: Pixxel Controller Registered and Ready

1980-012-14:03:20.55281 CFE\_ES\_ExitApp: Application PIXXEL\_CTRL called

CFE\_ES\_ExitApp

EVS Port1 1980-012-14:03:20.55291 66/1/PIXXEL\_MAIN 1: Pixxel Main Orchestrator Activated

1980-012-14:03:21.05306 CFE\_ES\_ExitApp: Application PIXXEL\_MAIN called

CFE\_ES\_ExitApp

OS\_GenericSymbolLookup\_Impl():114:SBN\_UDP\_Ops: ./cf/pixxel\_c.so: undefined symbol:

SBN\_UDP\_Ops

OS\_GenericSymbolLookup\_Impl():114:SBN\_UDP\_Ops: ./cf/pixxel\_main.so: undefined symbol:

SBN\_UDP\_Ops

OS\_GenericSymbolLookup\_Impl():114:SBN\_F\_Remap: ./cf/pixxel\_c.so: undefined symbol:

SBN\_F\_Remap

OS\_GenericSymbolLookup\_Impl():114:SBN\_F\_Remap: ./cf/pixxel\_main.so: undefined symbol:

SBN\_F\_Remap

EVS Port1 1980-012-14:03:21.15231 66/1/CFE\_ES 14: Exit Application PIXXEL\_CTRL Completed.

EVS Port1 1980-012-14:03:21.15245 66/1/CFE\_ES 14: Exit Application PIXXEL\_MAIN Completed